SafePlanner: Verification-Guided Staged Coordination for Safe Network Control

Zijian Lu*, Siqi Gong†, Yiping Zuo‡§, Xin He‡, Weibei Fan‡¶‖
* College of Telecommunications and Information Engineering,
Nanjing University of Posts and Telecommunications, Nanjing 210023, China
† College of Chemistry and Life Sciences, Nanjing University of Posts and Telecommunications, Nanjing 210023, China
‡ College of Computer, Nanjing University of Posts and Telecommunications, Nanjing 210023, China
§ National Mobile Communications Research Laboratory, Southeast University, Nanjing 210096, China
¶ State Key Laboratory of Internet Architecture, Tsinghua University, Beijing 100084, China
‖ State Key Lab of Processors, Institute of Computing Technology, CAS, China
Emails: 18818732360@163.com, b24070127@njupt.edu.cn,
zuoyiping@njupt.edu.cn, xhe@njupt.edu.cn, wbfan@njupt.edu.cn

***Abstract*—In modular control for multi-service wireless networks, a main challenge is to coordinate conflicting decisions from the ultra-reliable low-latency communication (URLLC) delay protection module and the enhanced mobile broadband (eMBB) throughput retention module without harming system stability. Direct one-shot arbitration may cause unsafe delay increases, throughput degradation, and configuration oscillation. To address this issue, we propose SafePlanner, a verification-guided coordination framework that constructs a compact staged control descriptor and unifies safety verification, iterative refinement, and conservative fallback within a single decision loop. Simulation results indicate that SafePlanner remains competitive with strong baselines in safety, while achieving higher average eMBB throughput and markedly lower online verification overhead.**

***Index Terms*—URLLC, eMBB , wireless network control, safe deployment.**

I. Introduction

The convergence of 6G and artificial intelligence is driving wireless network control toward more adaptive, service-aware, and programmable architectures. Instead of relying on a single monolithic optimizer, modern wireless systems increasingly adopt modular control stacks, where different control components specialize in different service objectives and can be updated independently as traffic patterns, deployment policies, and service demands evolve [1]–[3]. This architectural shift is further reinforced by Open Radio Access Network and slicing aware designs, which promote functional disaggregation and flexible control interfaces across the network stack [2], [3]. While such modularity improves flexibility and scalability, it also changes the core difficulty of control. The coordinator no longer needs only to optimize a global objective. It must also reconcile multiple locally reasonable recommendations that compete for the same radio resources.

This challenge becomes particularly clear under ultra-reliable low-latency communication (URLLC) and enhanced mobile broadband (eMBB) coexistence. URLLC protection emphasizes strict delay control, whereas eMBB performance relies on retaining broadband capacity [4], [5]. Prior studies on network slicing and resource allocation have clarified this tension at the system level [6], [7]. However, the problem is more delicate than static allocation in online operation. A latency oriented module may request stronger protection when delay approaches its boundary, while a throughput oriented module may request relaxation when broadband capacity begins to drop. Each recommendation is reasonable from its own service view, yet an aggressive response to either side can create a new violation for the other.

More importantly, this conflict has an inherent temporal attribute. Near a service boundary, the right decision is often not a hard choice between two modules. The coordinator may need to lean toward one service only slightly, postpone a strong intervention by one step, or test a milder adjustment before committing to a larger shift. These choices matter in closed loop wireless control. A strong action may repair the current key performance indicator but induce oscillation later, while a delayed action may preserve smoothness but allow a deficit to accumulate. An effective coordinator therefore needs to express not only which service to favor, but also how strongly to move and when that move should take effect.

Recent learning based wireless network control methods improve adaptability under changing traffic conditions, including reinforcement learning approaches for URLLC and eMBB

Corresponding author: Yiping Zuo (email: zuoyiping@njupt.edu.cn).

This work was supported in part by the National Natural Science Foundation of China under Grant No. 62202237. Additional support was provided by the Open Project of the State Key Laboratory of Internet Architecture under Grant No. HLW2025MS24, by the State Key Lab of Processors, Institute of Computing Technology, CAS, under Grant No. CLQ202506, by the Open Research Fund of the National Mobile Communications Research Laboratory, Southeast University, under Grant No. 2026D02, and by the Open Research Fund of the State Key Laboratory of Millimeter Waves under Grant No. K202602-06.

coexistence, slicing, and broader resource management [8]–[10]. These studies provide strong motivation for data driven coordination. However, many controllers still represent the final decision as a single action at each step. This representation is compact, but it hides important coordination structure. It does not clearly expose service preference, intervention strength, or execution timing. When service modules disagree, such a compressed action view leaves little room for the system to express softer responses, delayed responses, or probe style responses.

Safety aware control provides another important ingredient. Recent safe learning surveys, state wise safe learning, and constrained safe reinforcement learning show that learned decisions should be checked against operational constraints before deployment [11]–[13]. This perspective is especially relevant in artificial intelligence enabled wireless systems, where control decisions must remain inspectable and reliable [14], [15]. Yet a pure accept or reject view is still limited for modular multi service coordination. If every risky candidate is simply blocked, the system either falls back too often to conservative operation or pays a high online cost to search many alternatives. In this setting, verification is more useful when it not only filters decisions, but also provides structured signals that can guide a nearby correction.

Motivated by these observations, we study safe multi service coordination through an intermediate staged decision object rather than direct winner selection among controller outputs. The key idea is to separate service bias, actuation strength, and execution timing before an action is deployed. Such a representation gives the verifier a more meaningful object to inspect and gives the coordinator a local structure that can be revised when a candidate appears unsafe. Instead of discarding the original service intention immediately, the system can soften, rebalance, or defer the candidate within a bounded coordination loop.

Based on this view, we propose SafePlanner, a verification guided staged coordination framework for safe network control. SafePlanner first produces a lightweight coordination descriptor, then maps it to an executable action, evaluates one step risk, and applies bounded repair before resorting to conservative fallback. In this way, the proposed design preserves the modularity of service specific control while making the coordination layer more interpretable, more repairable, and more suitable for safe online deployment. The main contributions of this paper are as follows.

- We formulate safe URLLC and eMBB coordination as lightweight staged planning over service bias, actuation strength, and execution timing, which better matches online wireless control under conflicting recommendations.
- We design a verification-guided coordination loop that maps each staged descriptor to an executable action, predicts one-step risk, repairs unsafe candidates, and applies conservative fallback after bounded repair.
- We evaluate the proposed framework in closed loop simulations under representative traffic regimes. The results show that it achieves a favorable tradeoff among safety, throughput, and online verification cost.

II. System Model and Problem Formulation

We consider a slotted radio control system over a finite horizon $\mathcal{T} = \{0, 1, \ldots, T\}$. Fig. 1 gives an overview of the staged coordination pipeline. At each slot, the network serves URLLC and eMBB traffic with arrivals $\lambda_{u,t}$ and $\lambda_{b,t}$, allocates a normalized resource fraction $\alpha_t \in [\alpha_{\min}, \alpha_{\max}]$ to URLLC, and applies one of three scheduler modes

$$m_t \in \mathcal{M} \triangleq \{\mathrm{N}, \mathrm{P}, \mathrm{C}\}, \tag{1}$$

where N, P, and C denote the normal, protection, and conservative modes. Together with α_t, the scheduler mode determines the effective service rates

$$\mu_{x,t} = \begin{cases} \alpha_t \eta_u g_u(m_t), & x = u, \\ (1-\alpha_t)\eta_b g_b(m_t), & x = b, \end{cases} \qquad x \in \{u, b\}, \tag{2}$$

where η_u and η_b are per-resource service efficiencies and $g_u(\cdot)$ and $g_b(\cdot)$ are mode-dependent service factors.

Let $q_{u,t}$ and $q_{b,t}$ denote the URLLC and eMBB queue lengths. Their queue dynamics follow

$$q_{x,t+1} = \max\{0, q_{x,t} + \lambda_{x,t} - \mu_{x,t}\} + \xi_{x,t}, \tag{3}$$

where $\xi_{x,t}$ captures small unmodeled disturbance. From this operating point, we form the KPIs used by both the deployed modules and the safety checker. The URLLC delay surrogate is

$$d_{u,t} = \frac{1}{\max(\epsilon, \mu_{u,t} - \lambda_{u,t})}, \tag{4}$$

the eMBB throughput is

$$R_{b,t} = \min\{\lambda_{b,t}, \mu_{b,t}\}, \tag{5}$$

the normalized load pressure is

$$\rho_t = \min\left\{1, \frac{\lambda_{u,t} + \lambda_{b,t}}{\mu_{u,t} + \mu_{b,t} + \epsilon}\right\}, \tag{6}$$

the action variation is

$$\Delta a_t = |\alpha_t - \alpha_{t-1}| + \mathbb{I}\{m_t \neq m_{t-1}\}, \tag{7}$$

and the smoothed oscillation indicator is

$$o_t = \beta o_{t-1} + (1-\beta)\Delta a_t. \tag{8}$$

These quantities summarize current load pressure and reconfiguration smoothness. The state builder then aggregates the current loads, queue states, KPIs, operating point, previous control signature, and recent trend summary into

$$\mathbf{s}_t = [\lambda_{u,t}, \lambda_{b,t}, q_{u,t}, q_{b,t}, d_{u,t}, R_{b,t}, \alpha_t, m_t, \rho_t, o_t, \mathbf{c}_{t-1}, h_t]. \tag{9}$$

Here $\mathbf{c}_{t-1}$ records the latest executed control signature, while h_t summarizes recent KPI trends.

The network is assumed to already contain two deployed service specific modules. The URLLC delay protection module outputs

$$u_t^{(L)} = \begin{cases} P_2, & d_{u,t} > \vartheta_u^{(2)} D_u^{\max}, \\ P_1, & d_{u,t} > \vartheta_u^{(1)} D_u^{\max}, \\ S, & \text{otherwise}, \end{cases} \tag{10}$$

and the eMBB throughput retention module outputs

$$u_t^{(T)} = \begin{cases} R_2, & R_{b,t} < \vartheta_b^{(2)} R_b^{\min}, \\ R_1, & R_{b,t} < \vartheta_b^{(1)} R_b^{\min}, \\ S, & \text{otherwise}. \end{cases} \tag{11}$$

Here P_1 and P_2 denote low and high protection requests, R_1 and R_2 denote low and high relaxation requests, and S is the keep action. The trigger factors satisfy $0 < \vartheta_u^{(1)} < \vartheta_u^{(2)} < 1$ and $1 < \vartheta_b^{(2)} < \vartheta_b^{(1)}$, so P_2 and R_2 represent the more urgent interventions. The coordination problem is to reconcile these signals into an executable reconfiguration by pursuing

$$\max_{\{\mathbf{a}_t\}_{t=1}^T} \sum_{t=1}^{T} (\omega_b R_{b,t} - \omega_d d_{u,t} - \omega_o o_t - \omega_s |\Delta a_t|), \tag{12}$$

where the hard latency, throughput, load, and oscillation limits are enforced by the verifier.

Instead of selecting a raw action in one step, the coordinator first emits a staged descriptor

$$\mathbf{p}_t = (z_t, \ell_t, \tau_t), \tag{13}$$

which encodes coordination bias, actuation strength, and staging choice before the final executable action is determined. The three descriptor components are drawn from $\mathcal{Z} = \{\mathrm{L}, \mathrm{T}, \mathrm{B}, \mathrm{H}\}$ for favor latency, favor throughput, balanced, and hold, $\mathcal{A}_\ell = \{1, 2, 3\}$ for low, medium, and high actuation strength, and $\mathcal{T}_{\mathrm{cmd}} = \{\mathrm{I}, \mathrm{D}, \mathrm{P}\}$ for immediate execution, one-step deferral, and probing execution. Accordingly, the descriptor space is $\mathcal{P} = \mathcal{Z} \times \mathcal{A}_\ell \times \mathcal{T}_{\mathrm{cmd}}$.

The proposer generates an initial descriptor from the current operating context. At slot t, it forms an input vector $\mathbf{x}_t$ from the normalized state, the two module outputs, the previous feedback summary $\mathbf{f}_{t-1}$, the previous executed-action summary $\mathbf{c}_{t-1}$, and the previous fallback flag $\mathbb{I}_{t-1}^{\mathrm{fb}}$. The recurrent state is then updated as

$$\mathbf{r}_t = \mathrm{GRU}(\phi(\mathbf{x}_t), \mathbf{r}_{t-1}), \qquad \mathbf{r}_0 = \mathbf{0}, \tag{14}$$

The model then decodes $\mathbf{r}_t$ into the initial proposal $\mathbf{p}_t^{(0)}$. The descriptor is mapped into a candidate next-step control move by translating its bias, strength, and timing into a resource adjustment and next mode. The nominal step size associated with the strength level is

$$\delta(\ell_t) = \begin{cases} \delta_1, & \ell_t = 1, \\ \delta_2, & \ell_t = 2, \\ \delta_3, & \ell_t = 3, \end{cases} \tag{15}$$

where the default simulator uses $(\delta_1, \delta_2, \delta_3) = (0.03, 0.06, 0.10)$. The timing modifier is

$$\bar{\delta}(\ell_t, \tau_t) = \begin{cases} \delta(\ell_t), & \tau_t = \mathrm{I}, \\ 0, & \tau_t = \mathrm{D}, \\ \delta_1, & (\ell_t, \tau_t) \in \{(1, \mathrm{P}), (2, \mathrm{P})\}, \\ \delta_2, & (\ell_t, \tau_t) = (3, \mathrm{P}), \end{cases} \tag{16}$$

so probe applies a reduced intervention while defer keeps the current configuration in the present step. For balanced coordination, the resource direction is determined by the urgency gap

$$\kappa_t = \mathrm{clip}\left(\frac{d_{u,t}}{D_u^{\max}} - \frac{R_b^{\min}}{R_{b,t} + \epsilon}, -1, 1 \right). \tag{17}$$

Therefore, the resource update is

$$\Delta\alpha_t = \begin{cases} \bar{\delta}(\ell_t, \tau_t), & z_t = \mathrm{L}, \\ -\bar{\delta}(\ell_t, \tau_t), & z_t = \mathrm{T}, \\ \kappa_t \bar{\delta}(\ell_t, \tau_t), & z_t = \mathrm{B}, \\ 0, & z_t = \mathrm{H}. \end{cases} \tag{18}$$

The candidate resource split is

$$\alpha_{t+1}^{\mathrm{cand}} = \Pi_{[\alpha_{\min}, \alpha_{\max}]} (\alpha_t + \Delta\alpha_t), \tag{19}$$

and the next mode is

$$m_{t+1}^{\mathrm{cand}} = \begin{cases} \mathrm{P}, & z_t = \mathrm{L} \text{ and } \tau_t \neq \mathrm{D}, \\ \mathrm{N}, & z_t = \mathrm{T} \text{ and } \tau_t \neq \mathrm{D}, \\ m_t, & z_t \in \{\mathrm{B}, \mathrm{H}\} \text{ or } \tau_t = \mathrm{D}. \end{cases} \tag{20}$$

When $\tau_t = \mathrm{D}$, the current step keeps the existing configuration, and when $z_t = \mathrm{H}$, the action degenerates to $\Delta\alpha_t = 0$ with no mode change. The five atomic actions used by the service modules and the GA baseline are $P_1 = (\delta_1, \mathrm{P})$, $P_2 = (\delta_2, \mathrm{P})$, $R_1 = (-\delta_1, \mathrm{N})$, $R_2 = (-\delta_2, \mathrm{N})$, and $S = (0, m_t)$. This keeps GFP fair because it enumerates the same 36 descriptors used by SafePlanner and scores them with the same verifier.

Given $\mathbf{s}_t$ and a candidate descriptor, a one-step verifier predicts the next operating point and checks whether the resulting KPIs stay within the deployment limits. The one-step load predictor is

$$\hat{\lambda}_{x,t+1} = \lambda_{x,t} + \phi_x(\lambda_{x,t} - \lambda_{x,t-1}), \qquad x \in \{u, b\}, \tag{21}$$

where $\phi_x \in [0, 0.25]$ and $\hat{\lambda}_{x,t+1} = \lambda_{x,t}$ when $t = 0$. For $x \in \{u, b\}$, the predicted service rates are

$$\hat{\mu}_{x,t+1} = \begin{cases} \alpha_{t+1}^{\mathrm{cand}} \eta_u g_u(m_{t+1}^{\mathrm{cand}}), & x = u, \\ (1 - \alpha_{t+1}^{\mathrm{cand}}) \eta_b g_b(m_{t+1}^{\mathrm{cand}}), & x = b, \end{cases}. \tag{22}$$

The predicted URLLC delay reuses the inverse service-margin surrogate:

$$\hat{d}_{u,t+1} = \frac{1}{\max(\epsilon, \hat{\mu}_{u,t+1} - \hat{\lambda}_{u,t+1})}, \tag{23}$$

The predicted eMBB throughput is capped by predicted demand and service:

$$\hat{R}_{b,t+1} = \min\{\hat{\lambda}_{b,t+1}, \hat{\mu}_{b,t+1}\}, \tag{24}$$

The predicted load pressure normalizes total demand by total service:

$$\hat{\rho}_{t+1} = \min\left\{1, \frac{\hat{\lambda}_{u,t+1} + \hat{\lambda}_{b,t+1}}{\hat{\mu}_{u,t+1} + \hat{\mu}_{b,t+1} + \epsilon}\right\}, \tag{25}$$

The candidate variation records both resource movement and mode switching:

$$\Delta a_{t+1}^{\text{cand}} = \left|\alpha_{t+1}^{\text{cand}} - \alpha_t\right| + \mathbb{I}\{m_{t+1}^{\text{cand}} \neq m_t\}, \tag{26}$$

The predicted oscillation then updates the smoothed variation state:

$$\hat{o}_{t+1} = \beta o_t + (1-\beta)\Delta a_{t+1}^{\text{cand}}. \tag{27}$$

Together, these quantities let the verifier screen whether the candidate preserves latency, throughput, congestion, and smoothness within the next step. We define the normalized violation terms

$$\begin{cases} \delta_u = \max\left(0, \dfrac{\hat{d}_{u,t+1} - D_u^{\max}}{D_u^{\max}}\right), \\ \delta_b = \max\left(0, \dfrac{R_b^{\min} - \hat{R}_{b,t+1}}{R_b^{\min}}\right), \\ \delta_\rho = \max\left(0, \dfrac{\hat{\rho}_{t+1} - \rho_{\max}}{\rho_{\max}}\right), \\ \delta_o = \max\left(0, \dfrac{\hat{o}_{t+1} - o_{\max}}{o_{\max}}\right), \end{cases} \tag{28}$$

and the one step risk score

$$\mathcal{R}(\mathbf{s}_t, \mathbf{p}_t^{(k)}) = w_u\delta_u + w_b\delta_b + w_\rho\delta_\rho + w_o\delta_o + w_s|\Delta\alpha_t|. \tag{29}$$

The candidate is admitted only if all normalized violation terms are zero and the risk score is below threshold ϵ_R. When the candidate is rejected, the checker returns typed violation feedback, and the repair operator applies a bounded local update

$$\mathbf{p}_t^{(k+1)} = \mathcal{G}(\mathbf{s}_t, \mathbf{p}_t^{(k)}, v_t^{(k)}), \tag{30}$$

which adjusts only the bias, strength, or timing component and avoids full-plan search. If no repaired descriptor is admitted within K rounds, the controller enters the conservative branch

$$\mathbf{p}_t^{\text{fb}} = (\text{H}, 1, \text{I}). \tag{31}$$

This fallback descriptor is only a label for the conservative branch. Its executed action is defined directly rather than by reusing the nominal descriptor mapping, with $\Delta\alpha_t = 0$ and $m_{t+1} = \text{C}$. Once a candidate is admitted, the safe executor applies

$$a_t = (\Delta\alpha_t, m_{t+1}). \tag{32}$$

III. Numerical Results

We evaluate the proposed method on a closed loop simulation platform with four traffic regimes, namely normal load, URLLC burst, eMBB heavy load, and conflict-heavy load. Each episode lasts 200 control steps, and each method is evaluated over 10 random seeds. The simulation and control settings are listed in Table I. The descriptor space contains

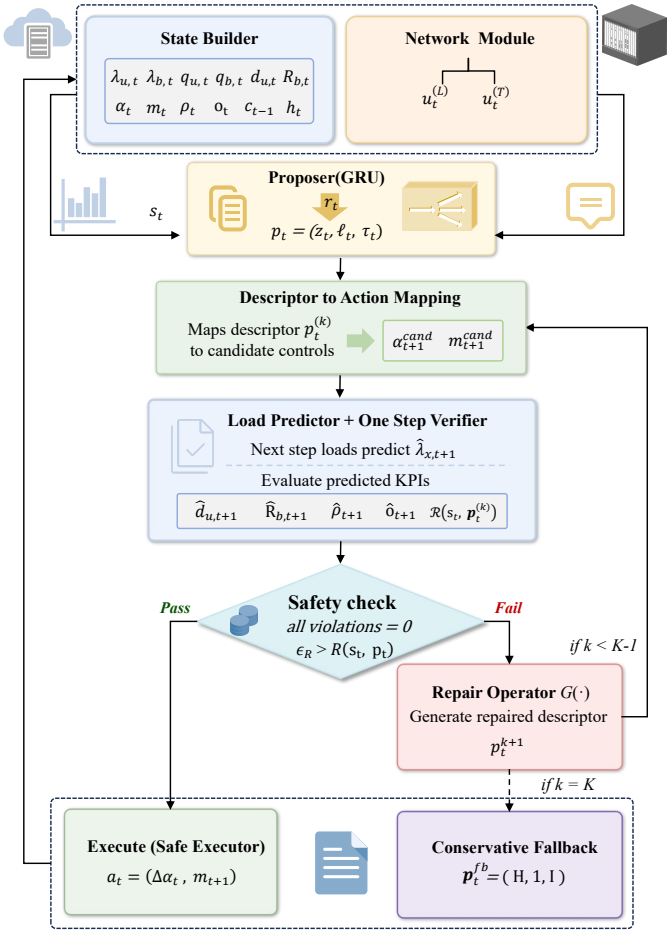


Fig. 1. System architecture for staged coordination. The blocks show the state, candidate control, predicted KPIs, repair loop, fallback branch, and executed action defined in the text.

Algorithm 1 Verification Guided Lightweight Coordination

Observe $\mathbf{s}_t$, $u_t^{(L)}$, $u_t^{(T)}$, and previous summaries $(\mathbf{r}_{t-1}, \mathbf{f}_{t-1}, \mathbf{c}_{t-1}, \mathbb{I}_{t-1}^{\text{fb}})$
Form $\mathbf{x}_t$ and update the recurrent state $\mathbf{r}_t$
Decode the initial plan $\mathbf{p}_t^{(0)} = (z_t, \ell_t, \tau_t)$
for $k = 0$ to $K-1$ **do**
  Run verifier on $\mathbf{p}_t^{(k)}$
  if plan passes **then**
    Map $\mathbf{p}_t^{(k)}$ to $a_t = (\Delta\alpha_t, m_{t+1})$
    Execute a_t and stop
  else if $k < K-1$ **then**
    Repair the plan using violation feedback
  end if
end for
Enter conservative safety branch and directly set $a_t = (0, \text{C})$
Execute a_t
Output: executed action a_t

36 candidates from 4 bias choices, 3 strength levels, and 3 timing choices, with repair budget $K = 3$ and verifier threshold $\epsilon_R = 0.12$. Before evaluation, the GRU proposer in SafePlanner is trained offline according to Table II and then fixed during testing, so the comparison focuses on the online coordination mechanism. We report six retained metrics. The unsafe rate is the fraction of executed control steps for which

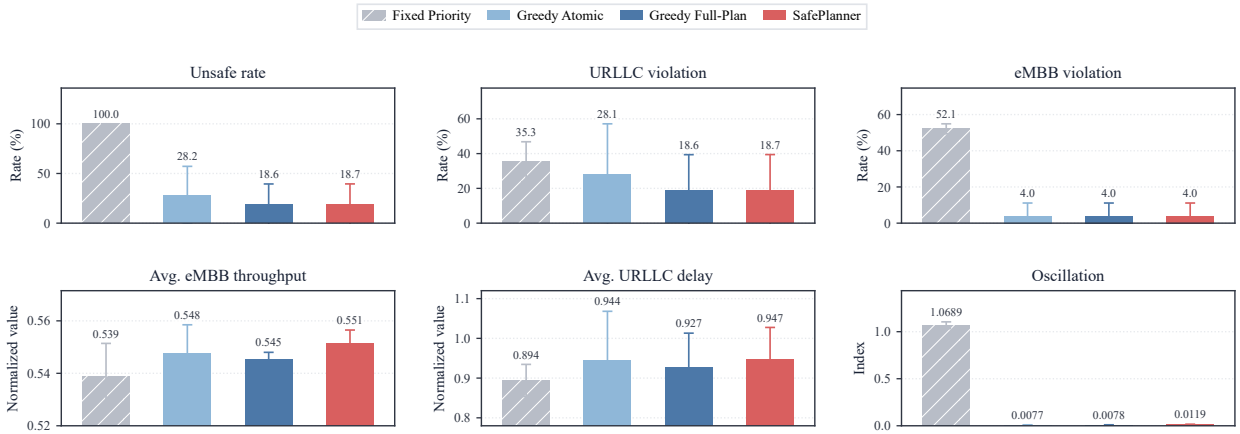


Fig. 2. Aggregate comparison over 10 seeds across the six retained metrics. SafePlanner remains essentially matched with Greedy Full-Plan on aggregate safety while achieving slightly higher average eMBB throughput.

at least one deployment limit is violated after execution, namely $d_{u,t} > D_u^{\max}$, $R_{b,t} < R_b^{\min}$, $\rho_t > \rho_{\max}$, or $o_t > o_{\max}$. GA, GFP, and SafePlanner are the verifier-based methods and therefore use the same verifier interface and the same conservative fallback branch. Fixed Priority executes the URLLC delay protection module whenever $u_t^{(L)} \neq S$ and otherwise applies the eMBB throughput retention module. Greedy Atomic screens the atomic action set $\{P_1, P_2, R_1, R_2, S\}$ and deploys the admissible action with the lowest predicted risk. Greedy Full-Plan exhaustively screens all 36 descriptors and selects the admissible descriptor with the lowest predicted risk. GFP is therefore the main comparison object because it provides the closest descriptor-level reference to SafePlanner while retaining exhaustive verifier screening.

Fig. 2 reports the aggregate comparison across the six retained metrics. The two weaker baselines first show that this problem cannot be handled well by simple one-step rules. Compared with FP, SafePlanner reduces the unsafe rate from 100.00% to 18.69%, the URLLC violation rate from 35.31% to 18.66%, and the eMBB violation rate from 52.14% to 4.02%. Compared with GA, SafePlanner also achieves a clearly better operating point, which indicates that screening atomic actions is still insufficient when the coordinator must jointly reason about service bias, intervention strength, and execution timing. The main result is the comparison with GFP.

TABLE I
SIMULATION AND CONTROL SETTINGS USED IN THE URLLC AND EMBB EVALUATION.

Parameter	Value
Episode length	200 control steps
Seeds	10
URLLC thresholds	$(0.80D_u^{\max},\ 0.95D_u^{\max})$
eMBB thresholds	$(1.20R_b^{\min},\ 1.05R_b^{\min})$
Actuation levels	$(\delta_1, \delta_2, \delta_3) = (0.03, 0.06, 0.10)$
Oscillation range	$\beta \in (0, 1)$
Predictor range	$\phi_x \in [0, 0.25]$
Repair budget	$K = 3$
Verifier threshold	$\epsilon_R = 0.12$

TABLE II
OFFLINE TRAINING CONFIGURATION FOR THE GRU-BASED PROPOSER.

Parameter	Value
Warm-start traces	20,000
Teacher episodes	120
Warm-start epochs	30
Warm-start batch	256
Warm-start lr	3×10^{-4}

SafePlanner remains essentially matched with GFP on aggregate safety, with unsafe rate 18.69% versus 18.65%, URLLC violation 18.66% versus 18.62%, and identical eMBB violation 4.02%. Under this nearly unchanged safety profile, SafePlanner improves average eMBB throughput from 0.5455 to 0.5514. The cost is a small increase in average URLLC delay, from 0.9267 to 0.9465, and oscillation, from 0.0078 to 0.0119. This result suggests that the learned staged coordinator is slightly more active in retaining broadband throughput, while the verifier and repair loop keep the safety profile close to exhaustive descriptor screening.

Fig. 3 explains why SafePlanner can approach GFP without paying the cost of exhaustive search. The baselines do not include iterative repair, so their repair success is zero, whereas SafePlanner repairs 56.76% of initially rejected descriptors. This shows that verifier feedback is not merely a rejection signal. It can guide an unsafe candidate toward a nearby deployable descriptor. More importantly, this efficiency does not come from weaker conservatism. SafePlanner has a fallback rate of 18.85%, which is very close to GFP at 18.60%, but it uses only 1.4851 verifier calls per step on average, while GFP uses 36 calls because it screens the whole descriptor space. Therefore, the main systems advantage of SafePlanner is that it preserves nearly the same deployment safety as GFP while greatly reducing online verification overhead.

Table III further shows which part of the sequential coordinator matters once the verifier and repair loop are fixed. Removing recurrence is the most damaging change: the unsafe rate rises from 18.69% to 37.40%, while repair success

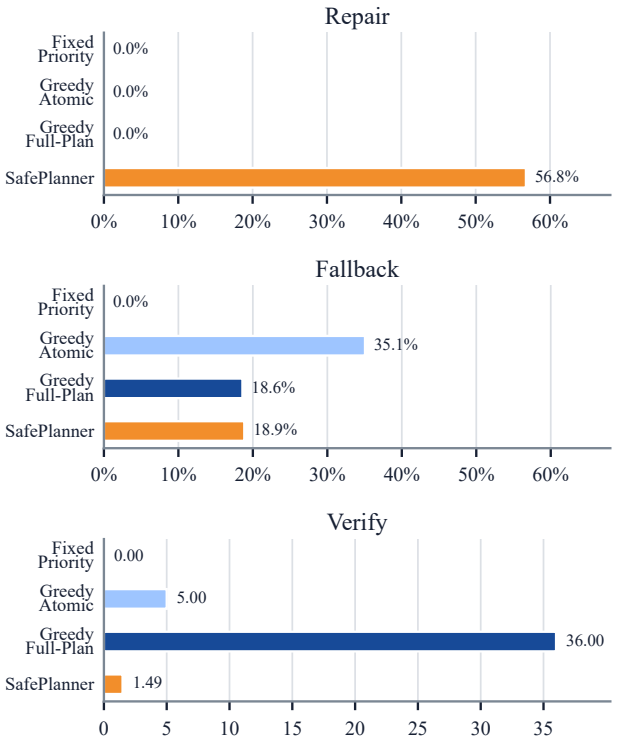


Fig. 3. Repair and verification cost comparison. SafePlanner alone achieves nonzero repair success while keeping fallback close to Greedy Full-Plan and verifier usage much lower.

TABLE III
ABLATION OF THE SEQUENTIAL COORDINATOR. VALUES ARE MEAN RESULTS OVER 10 SEEDS.

Variant	Unsafe (%)	Throughput	Osc.	Repair (%)
SafePlanner	18.69	0.5514	0.0119	56.76
No feedback	19.53	0.5544	0.0093	43.64
No recurrent proposer	28.16	0.5601	0.0078	60.49
No recurrence	37.40	0.5504	0.0090	26.61

drops from 56.76% to 26.61%. This indicates that short-term coordination history is critical for stable staged decisions near safety boundaries. Removing verifier feedback causes a smaller degradation in unsafe rate, from 18.69% to 19.53%, but reduces repair success to 43.64%, which suggests that feedback mainly improves how repairable the next descriptor is. The variant without recurrent proposer also degrades safety, with unsafe rate increasing to 28.16%. Taken together, these results show that the gain of SafePlanner does not come from the verifier alone, but from the combination of learned recurrent proposal, staged descriptor generation, and bounded repair under safety checking.

IV. CONCLUSION

We studied safe URLLC and eMBB coordination under conflicting local recommendations. The main message is that this problem should not be framed as direct winner selection among controller outputs. It should be framed as lightweight staged planning over bias, strength, and timing. With verifier guided repair, this representation keeps safety essentially matched with exhaustive plan screening, yields slightly higher eMBB throughput, and greatly reduces online verification cost. The broader implication is that multi-service wireless control becomes more deployable when the coordination object is interpretable, staged, and verifiable before execution.

REFERENCES

[1] W. Wu, C. Zhou, M. Li, H. Wu, H. Zhou, N. Zhang, X. S. Shen, and W. Zhuang, "AI-native network slicing for 6G networks," *IEEE Wireless Commun.*, vol. 29, no. 1, pp. 96–103, Feb. 2022.

[2] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 2, pp. 1376–1411, Jun. 2023.

[3] K. Alam, M. A. Habibi, M. Tammen, D. Krummacker, W. Saad, M. Di Renzo, T. Melodia, X. Costa-Perez, M. Debbah, A. Dutta, and H. D. Schotten, "A comprehensive tutorial and survey of O-RAN: Exploring slicing-aware architecture, deployment options, use cases, and challenges," *IEEE Commun. Surveys Tuts.*, vol. 28, pp. 1637–1678, Feb. 2026.

[4] Y. Zhao, X. Chi, L. Qian, Y. Zhu, and F. Hou, "Resource allocation and slicing puncture in cellular networks with eMBB and URLLC terminals coexistence," *IEEE Internet Things J.*, vol. 9, no. 19, pp. 18 431–18 444, Oct. 2022.

[5] H. Peng, L.-C. Wang, and Z. Jian, "Data-driven spectrum partition for multiplexing URLLC and eMBB," *IEEE Trans. Cogn. Commun. Netw.*, vol. 9, no. 2, pp. 386–397, Apr. 2023.

[6] S. Ebrahimi, F. Bouali, and O. C. L. Haas, "Resource management from single-domain 5G to end-to-end 6G network slicing: A survey," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 4, pp. 2836–2866, 2024.

[7] S. K. Taskou, M. Rasti, and E. Hossain, "End-to-end resource slicing for coexistence of eMBB and URLLC services in 5G-advanced/6G networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 7, pp. 8015–8032, Jul. 2024.

[8] M. Alsenwi, N. H. Tran, M. Bennis, S. R. Pandey, A. K. Bairagi, and C. S. Hong, "Intelligent resource slicing for eMBB and URLLC coexistence in 5G and beyond: A deep reinforcement learning based approach," *IEEE Trans. Wireless Commun.*, vol. 20, no. 7, pp. 4585–4600, Jul. 2021.

[9] S. Kumar, S. K. Vankayala, D. Singh, I. Roy, B. P. S. Sahoo, S. Yoon, and I. S. Kanakaraj, "Dynamic network slicing using deep reinforcement learning," in *Proc. IEEE Int. Conf. Adv. Netw. Telecommun. Syst. (ANTS)*, Hyderabad, India, Dec. 2021, pp. 384–389.

[10] J. A. Hurtado Sanchez, K. Casilimas, and O. M. Caicedo Rendon, "Deep reinforcement learning for resource management on network slicing: A survey," *Sensors*, vol. 22, no. 8, p. 3031, Apr. 2022.

[11] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annu. Rev. Control Robot. Auton. Syst.*, vol. 5, no. 1, pp. 411–444, May 2022.

[12] W. Zhao, T. He, R. Chen, T. Wei, and C. Liu, "State-wise safe reinforcement learning: A survey," in *Proc. 32nd Int. Joint Conf. Artif. Intell. (IJCAI)*, Macau, SAR China, Aug. 2023, Aug. 2023, pp. 6814–6822.

[13] Z. Chen, T. Duan, X. Yang, and X. Gong, "Safe reinforcement learning with constraints: A survey," in *Proc. 4th Conf. Fully Actuated Syst. Theory Appl. (FASTA)*, Nanjing, China, Jul. 2025, pp. 718–723.

[14] B. Brik, H. Chergui, L. Zanzi, F. Devoti, A. Ksentini, M. S. Siddiqui, X. Costa-Perez, and C. Verikoukis, "Explainable AI in 6G O-RAN: A tutorial and survey on architecture, use cases, challenges, and future research," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 5, pp. 2826–2859, Oct. 2025.

[15] R. Soundrarajan, C. Fiandrino, M. Polese, S. D'Oro, L. Bonati, and T. Melodia, "On AI verification in open RAN," *IEEE Commun. Mag.*, vol. 64, no. 3, pp. 39–44, Mar. 2026.